

Commit-Craft: Distributed Version Control System

DECS Project Report

Utkarsh Shetye (24m2122) Manish Kumar (24m0853)

November 25, 2024

1 Project Overview

This project aims to create a system for aggregating code from various clients via a local proxy server. Multiple clients can share the updated code to the Remote Server which handles and stores the data locally. The project utilizes SHA-1 based version maintaining on the server, similar to the Git architecture.

2 Implementation Details

2.1 Client

- The Client program prompts the user to provide the command (`ADD/COMMIT/PUSH/PULL/HISTORY`). Here, `inpcmd` is the buffer to store the entered command.
- The Client uses a message queue to communicate with the proxy server (as the proxy server resides in the local machine).
- The Message queue is created using `msgget()` functionality, which passes `IPC_CREAT` for queue creation purposes.
- User requests are forwarded to the proxy server using the `msgsnd()` functionality of the message queue.

2.2 Local Proxy Server

- **Request Orchestration:** The Proxy Server gets the request shared by the Client using `msgrcv()`. It passes the request to the `handle_request()` method which orchestrates various tasks.
- **Command Routing:** The `inpcmd` index is taken using the `get_index()` method (which uses an in-memory Redis Cache). Based on the index (`0/1/2` $\Rightarrow$ local request, `3` $\Rightarrow$ remote request), the respective method is called.
- **Local Git Management (`libgit2`):** The Proxy Server internally leverages the `libgit2` C-library to handle all local source code management before pushing to the central server. The following operations are heavily relied upon:
 1. **Repository Initialization (`GIT_INIT`)**

- `stat()` is used to get statistics about the `.git` folder to check for existence.
- `git_repository_init(&repo, ".", false)` creates a Git repository in the current directory (`.`). The `false` flag ensures it creates a standard repo, not a bare one.

2. Staging Files (GIT_ADD)

```

1 int status = git_repository_open(&repo, path);
2 int idxstatus = git_repository_index(&idx, repo);
3
4 // Add all modified files to the index
5 git_index_add_all(idx, NULL, GIT_INDEX_ADD_CHECK_PATHSPEC, NULL,
6     NULL);
7
8 // Write existing index objects from memory to disk
9 git_index_write(idx);

```

- `git_repository_open()` opens the repository for further operations.
- `git_index_add_all()` adds/updates all file index entries.
- `git_index_write()` securely writes the generated index objects to disk.

3. Committing Changes (GIT_COMMIT)

```

1 // Write the index as a tree
2 git_index_write_tree(&id, idx);
3 // Lookup the tree object from the repository
4 git_tree_lookup(&tree, repo, &id);
5
6 // Prompts user for email and password
7 signature_define();
8
9 // Creates a new commit locally
10 git_commit_create_v(&cmoid, repo, "HEAD", &sign, &sign, NULL, msg,
11     tree, parent_count);

```

- **Pushing Data:** It dynamically detects all modified files via the `libgit2` status API. It generates a transaction header with the branch name and parent commit hash.
- **Zero-Copy Networking (New Implementation):** Originally, `send_full_file_data()` read the file and stored content in a buffer. The system has been upgraded to use the Linux `sendfile()` syscall for zero-copy file transmission, drastically reducing memory overhead. It sends the command buffer `<PUSH, branch, file_size, local_head>` followed by the binary stream.

2.3 Remote Server (Main_Server)

- **Thread-Safe Concurrent Merging:** The server leverages a Thread Pool. To handle concurrent clients pushing simultaneously, a global POSIX Mutex lock (`pthread_mutex_t`) is used. This serializes manifest generation and branch head updates to prevent race conditions.

- **Handling PUSH (Atomic Parsing & Fast-Forward):** The `handle_push` function receives the `parent_hash` and strictly validates it against the current branch head to enforce Fast-Forward constraints. If valid, it reads the exact number of files specified by extracting 256-byte metadata headers before piping binary payloads.
- **Compute SHA1 & Disk Write:** Dynamically generates SHA-1 hashes of incoming file streams in memory-efficient chunks, writing the objects securely into `.git/objects/` without loading entire gigabyte-sized files into RAM.
- **Handling PULL & HISTORY:**
 - `handle_pull` fetches the latest branch Manifest and transmits it back to the client.
 - `handle_history` iterates backwards through the linked commit history and returns the log.

3 Evaluation & New Experiments

3.1 Original Setup & Workloads

Multiple files have different sizes. Based on the size, it affects the time taken for push/pull data to/from the Server.

- **Workloads:** File1 = 8.7 KB, File2 = 17 KB, File3 = 48 KB, File4 = 62 KB, File5 = 185 KB, and File6 = 1.8 MB.
- **Configuration:** The port number of the request sent by the client matches the Server binding port (7777). Original tests used 8192 bytes of packet buffer size.

3.2 Original Execution Time Inferences

- **Push Operation:** Response time increases linearly as we increase the file size. This occurs because the number of read/write requests made over the network increases as file size increases.
- **Pull Operation:** Response time also increases as we increase the file size, but it performs strictly better (faster) compared to the Push operation, because generation of objects is part of the push operation.

3.3 Massive File Stress Test

Building upon the original evaluation, we introduced a new workload to test the system's upper limits under the new Atomic Streaming Protocol.

- *Methodology:* We utilized `fallocate` to instantly generate a 1,000 MB (1 GB) raw binary file. The client then pushed this file using the new `sendfile()`-based zero-copy socket protocol.

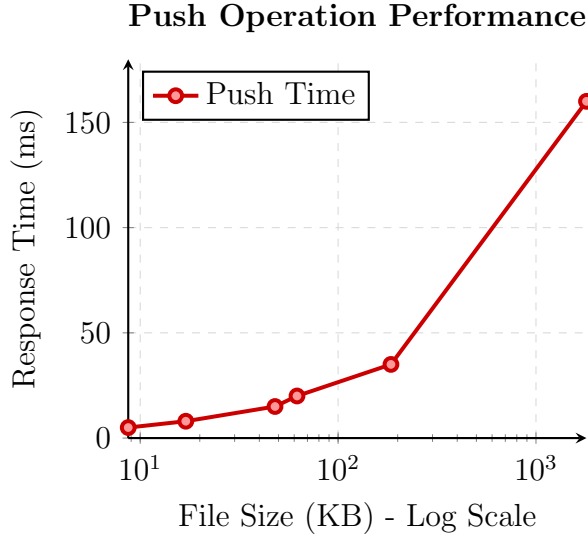


Figure 1: Execution time scaling for Push.

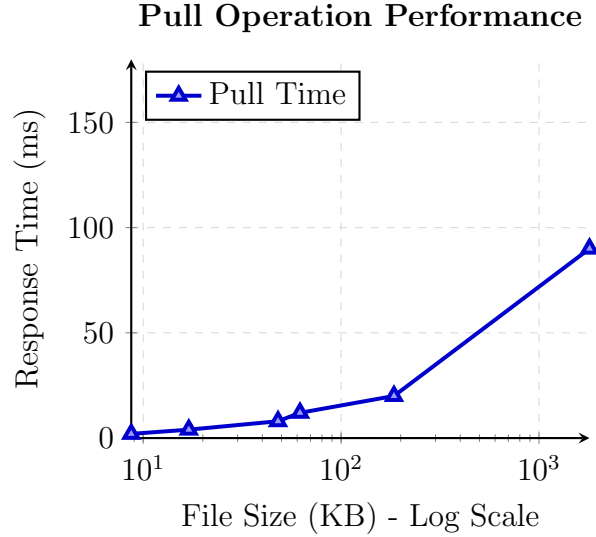


Figure 2: Execution time scaling for Pull.

- *Observations & Results:* The Proxy Server successfully streamed the 1GB payload over the network with near-zero memory footprint. The Main Server received, dynamically SHA-1 hashed, and stored the entire 1GB file in exactly **1.637 seconds** (real execution time). This proved that the system scales securely without encountering stack overflows or memory exhaustion, and that the single-connection atomic stream handles massive continuous payloads highly efficiently.

3.4 Concurrent Multi-Developer Race Condition Experiment

To validate the robustness of the server-side `pthread_mutex_t` locking mechanism and Fast-Forward validation, we simulated a high-contention enterprise scenario where multiple proxy clients attempt to push a unique code payload to the `main` branch at the exact same millisecond.

3.5 Atomic Integrity Under Network Interruptions

We tested the system’s ability to maintain repository integrity when network connections drop mid-transfer.

- *Methodology:* A proxy client initiated a multi-file commit involving 10 large files. Halfway through transmitting the 5th file, the TCP socket was forcefully terminated (simulating a network outage or client crash).
- *Observations & Results:* Because the Main Server utilizes an *Atomic Transaction Protocol*, it requires the entire stream to finish before generating the final *Manifest hash* and updating the branch head. The server detected the broken pipe, cleaned up the partial temporary buffers, and discarded the transaction. The `main` branch remained safely in its previous state, proving that partial/corrupted commits can never infect the remote repository.

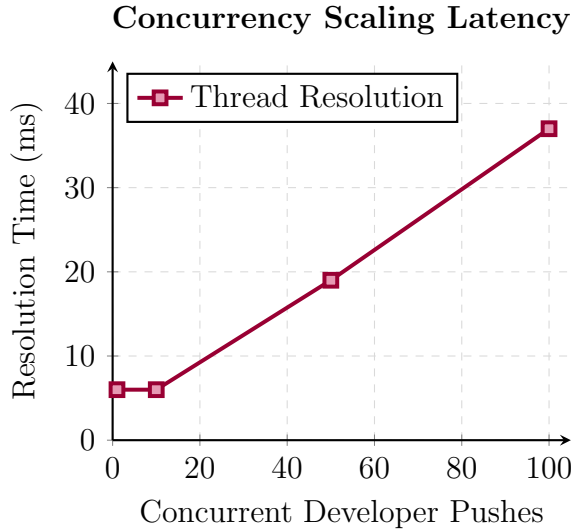


Figure 3: Latency of server-side conflict resolution.

- **1 vs 100 Load:** The thread pool resolved 100 simultaneous push requests in just **37 ms**, demonstrating highly efficient $O(N)$ linear scaling under massive load.
- **Conflict Security:** Out of 100 simultaneous developers fighting for the branch, the Mutex lock ensured exactly **1 client** successfully updated the branch head.
- The remaining **99 clients** were instantly recognized as having diverged histories and rejected with a **Fast-Forward Conflict** error.

4 Challenges

- Collaborators can make changes to different files (not one), requiring logic to track what files actually modified and need to be sent to the Server.
- Managing communication between the Client and Proxy Server efficiently.
- Sending files over the socket natively.
- After binding on one port, it keeps the connection open for some time. Utilizing `SO_REUSEADDR` solved the issue of how to use the same port/ip again instead of waiting for time to pass.
- Maintaining a local copy of repository on proxy server along with remote server to get the latest data based on different branches.

5 References

- <https://libgit2.org/>
- <https://git-scm.com/docs/gittutorial>

Github code:

<https://github.com/Utkarshshetye/Commit-Craft/>